Application of Natural Language Processing Towards Autonomous Software Testing

Khang Pham
Katalon Inc.
University of Science
Vietnam National University
Ho Chi Minh City, Vietnam
khang.pham@katalon.com

Vu Nguyen
Katalon Inc.
University of Science
Vietnam National University
Ho Chi Minh City, Vietnam
nvu@fit.hcmus.edu.vn

Tien Nguyen
Katalon Inc.
University of Texas at Dallas
Texas, USA
tien.n.nguyen@utdallas.edu

ABSTRACT

The process of creating test cases from requirements written in natural language (NL) requires intensive human efforts and can be tedious, repetitive, and error-prone. Thus, many studies have attempted to automate that process by utilizing Natural Language Processing (NLP) approaches. Furthermore, with the advent of massive language models and transfer learning techniques, people have introduced various advancements in NLP-assisted software testing with promising results. More notably, in recent years, not only have researchers been engrossed in solving the above task, but many companies have also embedded the feature to translate from human language to test cases their products. This paper presents an overview of NLP-assisted solutions being used in both the literature and the software testing industry.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**.

ACM Reference Format:
Khang Pham, Vu Nguyen, and Tien Nguyen. 2022. Application of Natural Language Processing Towards Autonomous Software Testing. In *37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22), October 10–14, 2022, Rochester, MI, USA.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3551349.3563241

1 INTRODUCTION

Testing has always been a fundamental activity in the life cycle of software, yet it is rather complex and effort-intensive. A testing procedure often consists of many phases, one of which is designing test cases from user requirements written in natural language [10]. NLP-assisted techniques are also used in other testing phases, e.g., test oracle generation, or web element identification. To reduce the effort of translating NL into test cases, several automated approaches based on NLP have been proposed and applied. Despite being only partially automated, they have saved testers and developers hours of manual work and potential human errors.

ASE '22, October 10–14, 2022, Rochester, MI, USA

ACM ISBN 978-1-4503-9475-8/22/10…$15.00
https://doi.org/10.1145/3551349.3563241

In this paper, similar to Garousi et al.[10], we use the phrase "NLP-assisted software testing" to refer to all NLP-based techniques and tools which could assist any software testing activity. In addition to advancements in the literature, NLP-assisted software testing has become a preeminent goal in the industrial product. Their benefits for enterprises are widely acknowledged, from improving testing efficiency to simplifying the testing process, which in turn increases automation coverage by allowing anyone to be involved in building tests. Henceforth, nowadays, being able to generate tests from English seems to become an important feature in every AI-powered testing tool. Given the large number of such tools, we believe surveying the industrial products along with advancements in the literature will greatly benefit practitioners and researchers.

This paper is structured as follows. In the next section, we outline several studies that are closely related to ours. In section 3, we introduce the basic concepts of NLP as well as current trends in the field. Next, in section 4, we discuss the role of NLP in autonomous testing, followed by a brief review of recent studies in NLP-assisted software testing in section 5. The industrial tools were mentioned and reviewed in section 6. Then, we point out several challenges of NLP-assisted software testing in section 7 before concluding in section 8.

2 RELATED WORK

As NLP application in software testing has been emerging over the past few years, a few papers focusing on this topic have been published. In a recent study [10], the authors conducted a survey including 67 research papers in the form of a systematic literature mapping focused on various techniques and research tools. Ahsan et al. [1] also investigated the application of NLP techniques and tools to generate test cases from the preliminary requirements document. Similarly, Gupta and Mahapatra [11] gave an in-depth comparative anatomization of studies that apply NLP to assist software testing in a circumstantial analysis. Another literature survey [5] outlined the state-of-the-art techniques in automatic test case generation; a few of them took the NLP approaches.

The above studies have investigated many NLP tools and ideas used in research, provided useful insights of how NLP is applied in test automation for practitioners and researchers. To the best of our knowledge, however, there has not been any related work taking into account the smart test automation tools in the industry with NLP-driven solutions. This paper is hoped to fulfill that missing piece and benefit enthusiasts in this topic.

3 AN OVERVIEW OF NLP

Natural language processing is a set of methods for making the human language accessible to computers [6]. Formally, it is a sub-field of computational linguistic and artificial intelligence (AI), covering a diverse range of tasks from examining rule-based syntax to modeling less intuitive semantic relationships. While the terms NLP and AI may arouse a feeling of futuristic and obscure inventions, they have actually been attached to our daily lives over the past few years. Google Translate - an application of machine translation, the e-mail spam filter utilizing text classification method, or the virtual assistant working based on speech processing and question-answering are just a few of the NLP applications.

Additionally, as mentioned earlier, advancements in NLP also contribute to accelerating the development of other research areas, including software testing, which is closely relevant to the information extraction task. Information extraction attempts to establish a robust way of summarizing and extracting relevant information from textual data, using a wide range of techniques such as tokenization, part-of-speech (POS) tagging, and dependency graph. For that purpose, it is the most widely used by both practitioners and researchers in the field of software engineering [11].

Another application of NLP which received much attention in recent years is language modeling. Language modeling is often used in sequence-to-sequence problems, where both the input and output are sequences of text, such as machine translation, speech recognition, and dialogue systems. The goal is to compute the probability of a sequence of words, which demonstrates the fluency of the output text given a problem-specific input. One notable thing about the language model is that once estimated, it can be reused in any application that generates English text, including generating test cases from natural language. This reusability is called transfer learning - one of the powerful factors of modern AI in general and NLP in particular.

4 THE ROLE OF NLP IN AUTONOMOUS TESTING

The benefits of automatic test case production are numerous, from saving time and effort test engineers put into tiresome procedures to avoiding potential human errors. Thus, this is an important research topic in the field of software engineering. In this section, we first give an overview of NLP-related problems in software testing. Then, we will discuss the scenarios where NLP approaches can be applied.

The research studies mainly aim to tackle these issues:

- **Problem modeling**: in software testing, problems with textual data (e.g., generating test cases from specifications in NL) are more complex than in traditional natural language processing. The NLP model not only has to understand the requirements in natural language but also must take into account relevant information of the system-under-test (SUT), which could be code or diagram. Thus, the problem is how to aggregate the two types of information without too much manual effort. This poses many challenges to the application of current NLP architectures in autonomous software testing.
- **Performance and accuracy**: the modern system requires test automation models to achieve a certain level of accuracy while being efficient enough for commercializing purposes. This is easier said than done, even with current NLP and autonomous testing development. The large language model alone is rather complicated (e.g., GPT-3 [4], a language model developed by OpenAI[1], has 175 billion parameters); now, with additional information from the SUT, it becomes even more complex. This complexity has made test automation a separate component of the development exercise. Scalability thus arises as an issue.
- **Input constraint**: most of the solution for automatic test generation from NLP, if not all, does not straight-up use natural language. They instead prefer controlled natural language (CNL), i.e., languages written in specific syntax rules [11]. CNL, on the one hand, can reduce the complexity and ambiguity of natural language, thus simplifying the test modeling process. On the other hand, it limits the freedom of natural language, making it less intuitive and requiring more manual work.

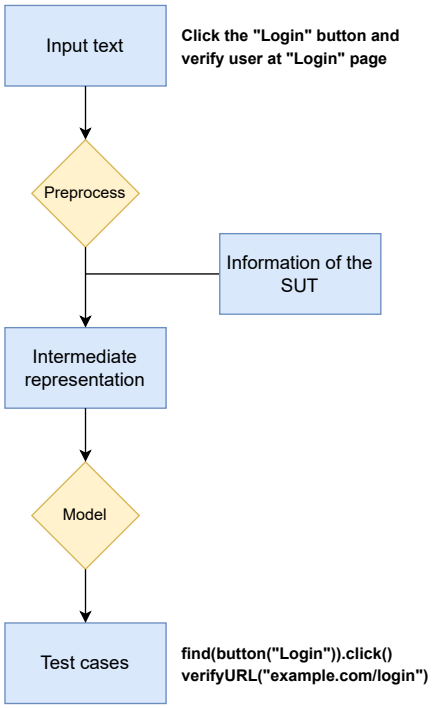

Figure 1: Example of an NLP approach for test generation

Most NLP-based approaches for test automation focus on the test design phase. Specifically, they require a given set of requirements written in natural language. These requirements will be preprocessed using proper NLP techniques; then, they will be combined with relevant information of the SUT to receive an intermediate representation, often in the form of a numerical vector. The test scripts will be extracted from such representation from a suitable NLP model. A similar process with minor modifications can also

[1] https://openai.com/

work to generate a test oracle from natural language input. The concept of this example is described in Figure 1.

5 ADVANCEMENTS IN RESEARCH

As the excellent assistance of NLP towards software testing has been proven, more and more researchers have dived into exploring that idea. Ansari et al. [2] has proposed a keyword-driven mechanism to extract test cases from software requirement specification (SRS). The authors reported that this approach, although being criticized as fundamental in nature [11], guarantees that all of the desiderata of the SRS are fully covered.

Another study [14] examined the test automation procedure under a specific environment comprising the Agile workflow. They took advantage of user stories, test scenes written in natural language, and acceptance criteria. They also introduced a dictionary containing keywords with their related test steps as an input parameter, which is new in test automation studies but has always been taken for granted in NLP tasks. Generally, this methodology does not rely on formal language. However, the author suggested that we should follow the provided instructions for writing test scenarios to avoid generating low-quality test cases.

The surveys for the above methodology, done by other researchers, indicate that there is still a great obstacle: the model did not generalize well with the diversity of functional requirement document (FRD), considering the distribution of development and testing groups and the size of software projects. Specifically, there should be a consistent comprehension of desiderata among various teams and individuals, which motivate researchers to dive in the direction of automated analysis of desiderata.

More recently, Wang et al. [17] showed positive results when using 5 NLP techniques: tokenization, POS tagging, NER, SRL, and semantic similarity detection. By incorporating new advances in NLP, they correctly generated 95% of the constraints required for testing the two systems in the case studies while requiring fewer efforts and resources than a manual process. Still, this approach is not independent of human involvement, as it requires a class diagram of the system under test to generate constraints that are used to generate test input data.

In UI testing, a recent study by [13] has proposed a script-free testing approach, leveraging NLP techniques to understand the semantics of web elements. The problem they tried to address is the fragile nature of the web element locator, which depends on the metadata and the structure of the web page. With the test written in a domain-specific language (which is close to NL), the idea is to first turn web elements and target operations into vectors in their semantic space. The vector representation of the web element consists of certain ambiguity; thus, they use heuristic search algorithms to find promising test procedures. The results opened up new possibilities for further research.

6 NLP SUPPORTED TESTING TOOLS

As manual testing is an exhaustive process, testers are open to a helping hand from any automated approaches. Thence, many intelligent testing companies have embedded NLP solutions in their products, providing users with a more straightforward test creation process. The below products are only a few examples of such tools.

6.1 Functionize

Functionize [8] is an intelligent software testing platform for machine learning and other AI-based technologies to reduce the time and cost spent on testing while accelerating product releases. They provide an NLP engine [9] that takes test plans written in plain English and generates fully-functional tests. This engine requires users to "treat the steps like you are writing them for a machine. Keep the instructions very simple and single-purposed". That means the plain English test step should follow a predefined set of rules and keywords in order to function correctly.

6.2 Microfocus's UFT One

UFT One [7] is a fully functional AI-driven test automation solution that allows customers to test earlier and more frequently. The AI capabilities of UFT One and UFT Developer are named Translator (NLP), Brain (Neural Networks), and Eyes (Computer Vision). With the Translator, users can reduce test creation time and simplify test maintenance by writing tests in almost plain English.

One distinguishing feature of the Translator is that it tolerates minor spelling mistakes in the input prompt. When identifying AI test objects, we can instruct UFT One to look specifically for the specified text if necessary.

6.3 Sauce Labs's AutonomIQ

AutonomIQ [3] is an AI-driven, autonomous low-code automation platform that is designed to help users achieve the highest quality outcome in the shortest amount of time possible. They introduce a Natural Language Engine that easily empowers anyone to create a Selenium script within minutes. To take advantage of it, one must provide the test steps, descriptions, and data (if any). From there, the test scripts can be automatically created, along with a screenshot with elements under test being highlighted.

AutonomIQ has provided a very detailed guide for its users[2]. There are 177 NLP commands available for testers. Although the command syntaxes are mostly fixed, the NLP engine accepts many synonyms. For instance, an action "enter some words" can be described using "enter," "fill in," "type in," or "set text."

6.4 Virtuoso

Virtuoso [16] combines Natural Language Programming, Machine Learning, and Robotic Process Automation in one platform to deliver test hyper-automation. The critical feature of Virtuoso is that they have bots that do all the work of test authoring. Generally, they allow users to use plain English to write tests that can be automated at the click of a button and executed immediately. Besides, they also allow users to write input texts similar to JavaScript expression, creating more robust requirements, despite being less intuitive.

7 CHALLENGES OF APPLYING NLP IN SOFTWARE TESTING

While the NLP model in other fields has made certain accomplishments, its impact on software testing may not live up to expectations. In the surveyed tools, features like smart locators and test generation from natural language may work, but they are pretty

[2]https://autonomiq.io/images/pdf/NLP_userGuide.pdf

limited and easy to break. Test oracle generation is still a new approach, and there has not been support for any other languages besides English.

From our observation, such under-performance could come from the complexity of software testing problems compared to traditional NLP. Unlike the traditional sequence-to-sequence problems, the output of test generation is written in a scripting language, which is stricter and less ambiguous than natural language. Most of the NLP models used in software testing, therefore, need a middle layer to convert the natural language into a intermediate representation (e.g., the UML diagram) in order to overcome the difference between the input and output languages. This could be the bottleneck of the whole procedure, as it may attenuate the effect of the vectorized representation adapting from a pre-trained model.

Another reason why software testing tools have not been so successful is the performance issues. We have mentioned massive language models with excellent results yet very expensive to build or use. If enterprises want to integrate such costly NLP engines into their products, they have to ensure that the software's performance is still good while trying to keep a competitive price. Finally, in our humble opinion, NLP alone is insufficient to make a good test design. Because natural language cannot help being ambiguous, the script generated by the NLP engine should be augmented by other AI techniques, for example, using Graph (e.g., RelicX [15]) or Computer Vision (e.g., Katalon Studio [12]).

8 CONCLUSION

Manual testing is an expensive and time-consuming software testing activity required to get consumer input on freshly released features. In this paper, we have discussed the scenarios and cases where NLP is being used or can be used in the software testing industry and demonstrated that using NLP in software testing not only reduces time but also increases the efficiency and quality of testing. We have also mentioned several intelligent testing tools in the industry to compare their capability with the advancements in the literature. When combined with smart models and algorithms, NLP has the ability to understand structured data efficiently. It can monitor and operate the vast majority of testing facilities, significantly improving testing outcomes and giving more reliable results in a timely way.

In this paper, we have presented an overview of the application of NLP in autonomous software testing, including recent advancements in the studies and several industrial tools. We have also mentioned a few challenges in the area. By that, we encourage researchers and practitioners to continue exploring such ideas and inventing new solutions for the software testing industry.

REFERENCES

[1] Imran Ahsan, Wasi Haider Butt, Mudassar Adeel Ahmed, and Muhammad Waseem Anwar. 2017. A comprehensive investigation of natural language processing techniques and tools to generate automated test cases. In *Proceedings of the Second International Conference on Internet of things, Data and Cloud Computing*. 1–10.

[2] Ahlam Ansari, Mirza Baig Shagufta, Ansari Sadaf Fatima, and Shaikh Tehreem. 2017. Constructing test cases using natural language processing. In *2017 Third International Conference on Advances in Electrical, Electronics, Information, Communication and Bio-Informatics (AEEICB)*. IEEE, 95–99.

[3] AutonomIQ. 2020. AutonomIQ | Home. Retrieved September 5, 2022 from https://katalon.com/katalon-studio/

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[5] Christian Denger and M Medina Mora. 2003. Test case derived from requirement specifications. *Fraunhofer IESE Report* (2003).

[6] Jacob Eisenstein. 2019. *Introduction to natural language processing*. MIT press.

[7] Micro Focus. 2022. Automate Functional Testing from UI to the API | UFT One | Micro Focus. Retrieved September 5, 2022 from https://www.microfocus.com/en-us/products/uft-one/overview

[8] Functionize. 2022. AI Test Automation with Machine Learning | Functionize. Retrieved September 5, 2022 from functionize.com

[9] Functionize. 2022. Natural Language Processing. Retrieved September 5, 2022 from https://www.functionize.com/natural-language-processing

[10] Vahid Garousi, Sara Bauer, and Michael Felderer. 2020. NLP-assisted software testing: A systematic mapping of the literature. *Information and Software Technology* 126 (2020), 106321.

[11] Atulya Gupta and Rajendra Prasad Mahapatra. 2021. A Circumstantial Methodological Analysis of Recent Studies on NLP-driven Test Automation Approaches. In *Intelligent Systems*. Springer, 155–167.

[12] Katalon. 2022. Katalon Studio. Retrieved September 5, 2022 from https://katalon.com/katalon-studio/

[13] Hiroyuki Kirinuki, Shinsuke Matsumoto, Yoshiki Higo, and Shinji Kusumoto. 2021. NLP-assisted Web Element Identification Toward Script-free Testing. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 639–643.

[14] Prerana Pradeepkumar Rane. 2017. *Automatic generation of test cases for agile using natural language processing*. Ph. D. Dissertation. Virginia Tech.

[15] RelicX. 2022. Relicx | Digital Experience Monitoring & Testing for DevOps. Retrieved September 5, 2022 from https://www.relicx.ai

[16] Virtuoso. 2022. QA Automation Testing Tools based on NLP, AI & Machine Learning | Virtuoso QA. Retrieved September 5, 2022 from https://www.virtuoso.qa/

[17] Chunhui Wang, Fabrizio Pastore, Arda Goknil, and Lionel Briand. 2020. Automatic generation of acceptance test cases from use case specifications: an nlp-based approach. *IEEE Transactions on Software Engineering* (2020).